[09-30-2026] BITS

September 30, 2026

CodeChum: BITS

Item1 - Bit Vector Activity

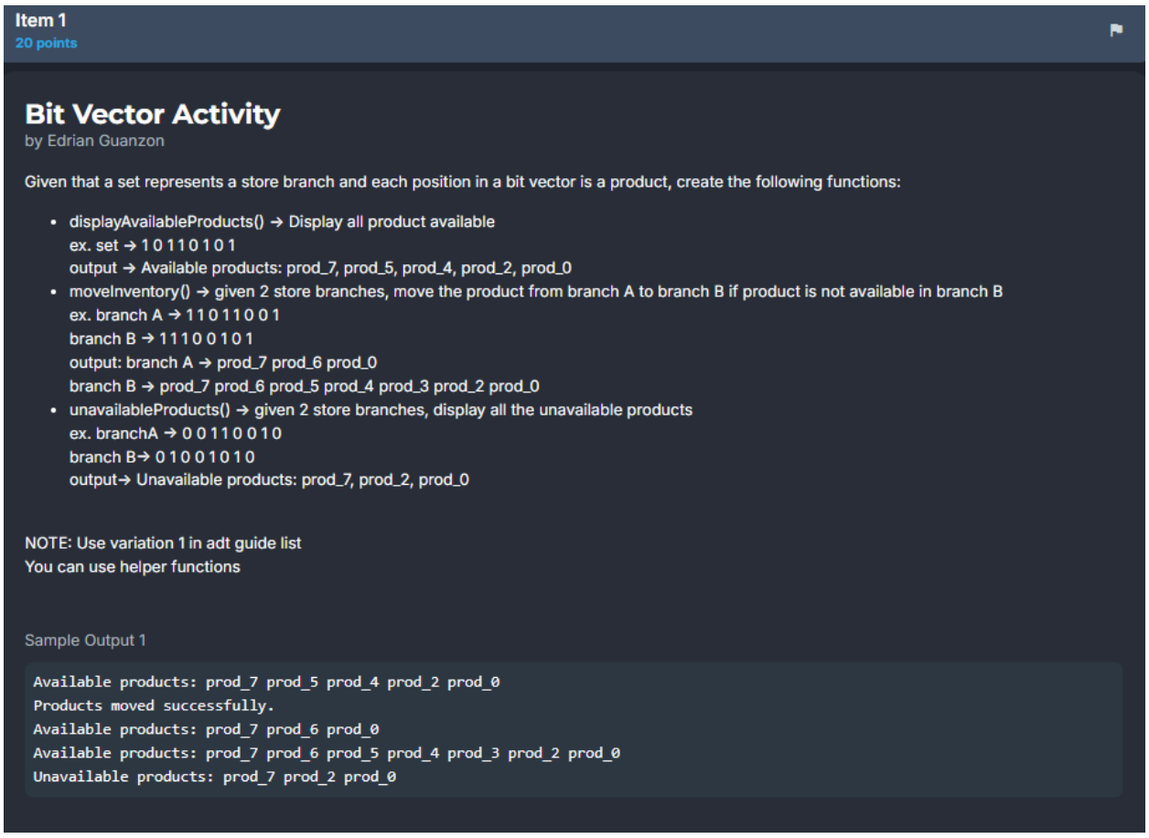
by Edrian Guanzon

Filename:

[windows] → *filename*

[macOS] → *filename*

Tasks/Instructions:



Item 1
20 points

Bit Vector Activity
by Edrian Guanzon

Given that a set represents a store branch and each position in a bit vector is a product, create the following functions:

- displayAvailableProducts() → Display all product available
  ex. set → 1 0 1 1 0 1 0 1
  output → Available products: prod_7, prod_5, prod_4, prod_2, prod_0
- moveInventory() → given 2 store branches, move the product from branch A to branch B if product is not available in branch B
  ex. branch A → 1 1 0 1 1 0 0 1
  branch B → 1 1 1 0 0 1 0 1
  output: branch A → prod_7 prod_6 prod_0
  branch B → prod_7 prod_6 prod_5 prod_4 prod_3 prod_2 prod_0
- unavailableProducts() → given 2 store branches, display all the unavailable products
  ex. branchA → 0 0 1 1 0 0 1 0
  branch B→ 0 1 0 0 1 0 1 0
  output→ Unavailable products: prod_7, prod_2, prod_0

NOTE: Use variation 1 in adt guide list
You can use helper functions

Sample Output 1

```
Available products: prod_7 prod_5 prod_4 prod_2 prod_0
Products moved successfully.
Available products: prod_7 prod_6 prod_0
Available products: prod_7 prod_6 prod_5 prod_4 prod_3 prod_2 prod_0
Unavailable products: prod_7 prod_2 prod_0
```

Test Cases:

item1

Code:

main.c - answer here (raw)

```c
#include <stdio.h>

void displayAvailableProducts(unsigned char products) {

}

void moveInventory(unsigned char *branchA, unsigned char *bra
nchB) {

}

void unavailableProducts(unsigned char branchA, unsigned char
branchB) {


}

int main() {
    unsigned char branch_A = 0;
    unsigned char branch_B = 0;

    branch_A |= (1 << 7);
    branch_A |= (1 << 5);
    branch_A |= (1 << 4);
    branch_A |= (1 << 2);
    branch_A |= (1 << 0);

    displayAvailableProducts(branch_A);

    branch_A = 0;
    branch_B = 0;

    branch_A |= (1 << 7);
    branch_A |= (1 << 6);
    branch_A |= (1 << 4);
    branch_A |= (1 << 3);
    branch_A |= (1 << 0);

    branch_B |= (1 << 7);
    branch_B |= (1 << 6);
    branch_B |= (1 << 5);
    branch_B |= (1 << 2);
    branch_B |= (1 << 0);

    moveInventory(&branch_A, &branch_B);
    displayAvailableProducts(branch_A);
    displayAvailableProducts(branch_B);
```

```
    branch_A = 0;
    branch_B = 0;

    branch_A |= (1 << 5);
    branch_A |= (1 << 4);
    branch_A |= (1 << 1);

    branch_B |= (1 << 6);
    branch_B |= (1 << 3);
    branch_B |= (1 << 1);

    unavailableProducts(branch_A, branch_B);

    return 0;
}
```

main.c - answered code

```
#include <stdio.h>

void printProducts(const char *label, unsigned char products)
{
    int i = 1 <<(sizeof(products)*8-1);
    int prod = sizeof(products) * 8 - 1;

    printf("%s", label);
    while(i>0){
        if(products & i){
            printf(" prod_%d", prod);
        }
        i = i >> 1;
        prod--;
    }
    printf("\n");
}

void displayAvailableProducts(unsigned char products) {
    printProducts("Available products:", products);
}

void moveInventory(unsigned char *branchA, unsigned char *bra
nchB) {
    unsigned char toMove = *branchA & ~(*branchB);
    *branchB |= toMove;
    *branchA &= ~toMove;
```

```
    printf("Products moved successfully.\n");
}

void unavailableProducts(unsigned char branchA, unsigned char
branchB) {
    unsigned char unavailable = ~(branchA | branchB);
    printProducts("Unavailable products:", unavailable);

}

int main() {
    unsigned char branch_A = 0;
    unsigned char branch_B = 0;

    branch_A |= (1 << 7);
    branch_A |= (1 << 5);
    branch_A |= (1 << 4);
    branch_A |= (1 << 2);
    branch_A |= (1 << 0);

    displayAvailableProducts(branch_A);

    branch_A = 0;
    branch_B = 0;

    branch_A |= (1 << 7);
    branch_A |= (1 << 6);
    branch_A |= (1 << 4);
    branch_A |= (1 << 3);
    branch_A |= (1 << 0);

    branch_B |= (1 << 7);
    branch_B |= (1 << 6);
    branch_B |= (1 << 5);
    branch_B |= (1 << 2);
    branch_B |= (1 << 0);

    moveInventory(&branch_A, &branch_B);
    displayAvailableProducts(branch_A);
    displayAvailableProducts(branch_B);

    branch_A = 0;
    branch_B = 0;

    branch_A |= (1 << 5);
    branch_A |= (1 << 4);
    branch_A |= (1 << 1);
```

```
    branch_B |= (1 << 6);
    branch_B |= (1 << 3);
    branch_B |= (1 << 1);

    unavailableProducts(branch_A, branch_B);

    return 0;
}
```

main.c - answered code (without using helper function)

```
/*Date: November 6, 2026
Status: Done
Instructions: Bit Vector Activity
    (Check Notion or CodeChum)

Notes/Comments/Feedback:
-It works but this doesn't have the helper function.
*/

//write code here
#include <stdio.h>

void displayAvailableProducts(unsigned char products) {
    int i = 1 << (sizeof(products) * 8 - 1);
    int prod = sizeof(products) * 8 - 1;

    printf("Available products:");

    while(i > 0){
        if(products & i){
            printf(" prod_%d", prod);
        }
        i = i >> 1;
        prod--;
    }
    printf("\n");

}

void moveInventory(unsigned char *branchA, unsigned char *bra
nchB) {
    unsigned char toMove = *branchA & ~(*branchB);

    *branchB |= toMove;
    *branchA &= ~toMove;
```

```
    printf("Products moved successfully.\n");
}

void unavailableProducts(unsigned char branchA, unsigned char
branchB) {
    unsigned char unavailable = ~(branchA |= branchB);

    int i = 1 << (sizeof(unavailable) * 8 - 1);
    int prod = sizeof(unavailable) * 8 - 1;

    printf("Unavailable products:");

    while(i > 0){
        if(unavailable & i){
            printf(" prod_%d", prod);
        }
        i = i >> 1;
        prod--;
    }
    printf("\n");

}

int main() {
    unsigned char branch_A = 0;
    unsigned char branch_B = 0;

    branch_A |= (1 << 7);
    branch_A |= (1 << 5);
    branch_A |= (1 << 4);
    branch_A |= (1 << 2);
    branch_A |= (1 << 0);

    displayAvailableProducts(branch_A);

    branch_A = 0;
    branch_B = 0;

    branch_A |= (1 << 7);
    branch_A |= (1 << 6);
    branch_A |= (1 << 4);
    branch_A |= (1 << 3);
    branch_A |= (1 << 0);

    branch_B |= (1 << 7);
    branch_B |= (1 << 6);
    branch_B |= (1 << 5);
```

```
    branch_B |= (1 << 2);
    branch_B |= (1 << 0);

    moveInventory(&branch_A, &branch_B);
    displayAvailableProducts(branch_A);
    displayAvailableProducts(branch_B);

    branch_A = 0;
    branch_B = 0;

    branch_A |= (1 << 5);
    branch_A |= (1 << 4);
    branch_A |= (1 << 1);

    branch_B |= (1 << 6);
    branch_B |= (1 << 3);
    branch_B |= (1 << 1);

    unavailableProducts(branch_A, branch_B);

    return 0;
}
```

Visualization:

Untitled

Feedback/comment:

Discussion/Explanation:

Item 2 - Questionnaire - CompWord

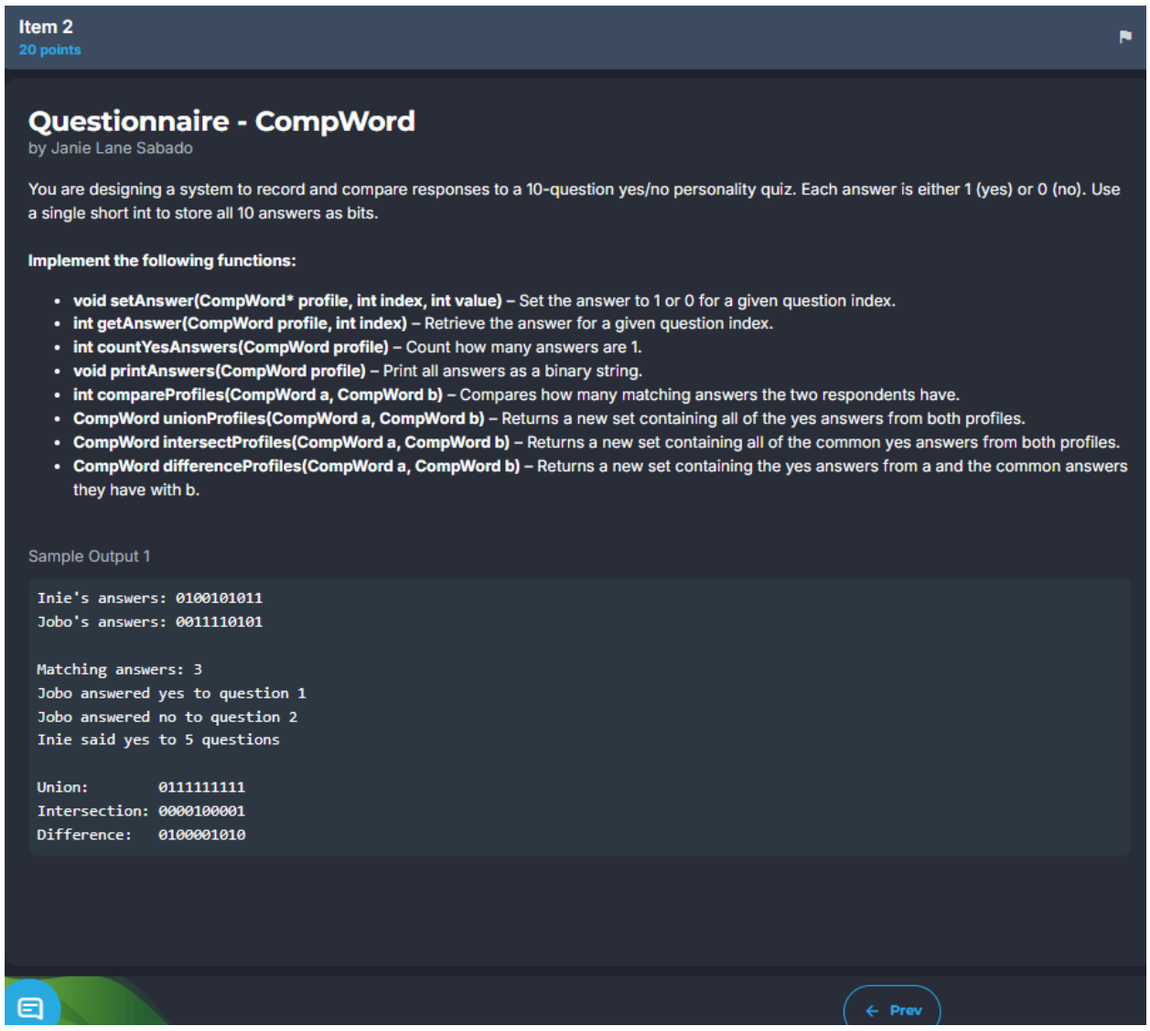
by Janie Lane Sabado

Filename:

[windows] → *filename*

[macOS] → *filename*

Tasks/Instructions:



Item 2
20 points

Questionnaire - CompWord

by Janie Lane Sabado

You are designing a system to record and compare responses to a 10-question yes/no personality quiz. Each answer is either 1 (yes) or 0 (no). Use a single short int to store all 10 answers as bits.

Implement the following functions:

- **void setAnswer(CompWord* profile, int index, int value)** – Set the answer to 1 or 0 for a given question index.
- **int getAnswer(CompWord profile, int index)** – Retrieve the answer for a given question index.
- **int countYesAnswers(CompWord profile)** – Count how many answers are 1.
- **void printAnswers(CompWord profile)** – Print all answers as a binary string.
- **int compareProfiles(CompWord a, CompWord b)** – Compares how many matching answers the two respondents have.
- **CompWord unionProfiles(CompWord a, CompWord b)** – Returns a new set containing all of the yes answers from both profiles.
- **CompWord intersectProfiles(CompWord a, CompWord b)** – Returns a new set containing all of the common yes answers from both profiles.
- **CompWord differenceProfiles(CompWord a, CompWord b)** – Returns a new set containing the yes answers from a and the common answers they have with b.

Sample Output 1

```
Inie's answers: 0100101011
Jobo's answers: 0011110101

Matching answers: 3
Jobo answered yes to question 1
Jobo answered no to question 2
Inie said yes to 5 questions

Union:        0111111111
Intersection: 0000100001
Difference:   0100001010
```

← Prev

Test Cases:

Item 2

Code:

main.c - locked

```c
#include <stdio.h>
#include "compword.h"

int main() {
    CompWord inie = 0;
    CompWord jobo = 0;

    setAnswer(&inie, 0, 1); // Q1: Yes
    setAnswer(&inie, 1, 1); // Q2: Yes
    setAnswer(&inie, 3, 1); // Q4: Yes
    setAnswer(&inie, 5, 1); // Q6: Yes
    setAnswer(&inie, 8, 1); // Q9: Yes
```

```
    setAnswer(&jobo, 0, 1); // Q1: Yes
    setAnswer(&jobo, 2, 1); // Q3: Yes
    setAnswer(&jobo, 4, 1); // Q5: Yes
    setAnswer(&jobo, 5, 1); // Q6: Yes
    setAnswer(&jobo, 6, 1); // Q7: Yes
    setAnswer(&jobo, 7, 1); // Q8: Yes

    printf("Inie's answers: ");
    printAnswers(inie);

    printf("Jobo's answers: ");
    printAnswers(jobo);

    printf("\nMatching answers: %d\n", compareProfiles(inie,
jobo));

    printf("Jobo answered %s to question %d\n",(getAnswer(job
o,0))?"yes":"no",1);
    printf("Jobo answered %s to question %d\n",(getAnswer(job
o,1))?"yes":"no",2);
    printf("Inie said yes to %d questions\n\n", countYesAnswe
rs(inie));

    CompWord unionSet = unionProfiles(inie, jobo);
    CompWord intersectSet = intersectProfiles(inie, jobo);
    CompWord diffSet = differenceProfiles(inie, jobo);

    printf("Union:        "); printAnswers(unionSet);
    printf("Intersection: "); printAnswers(intersectSet);
    printf("Difference:   "); printAnswers(diffSet);

    return 0;
}
```

compword.h - locked

```
#ifndef COMPWORD_H
#define COMPWORD_H

#include <stdio.h>
#include <stdbool.h>

typedef short int CompWord;

void setAnswer(CompWord* profile, int index, int value);
int getAnswer(CompWord profile, int index);
```

```c
int countYesAnswers(CompWord profile);
void printAnswers(CompWord profile);
int compareProfiles(CompWord a, CompWord b);

CompWord unionProfiles(CompWord a, CompWord b);
CompWord intersectProfiles(CompWord a, CompWord b);
CompWord differenceProfiles(CompWord a, CompWord b);


#endif // COMPWORD_H
```

compword.c - answer here (raw)

```c
#include <stdio.h>
#include <stdbool.h>
#include "compword.h"

// write your functions here...
```

compword.c - answered code

```c
#include <stdio.h>
#include <stdbool.h>
#include "compword.h"

// write your functions here...
void setAnswer(CompWord* profile, int index, int value) {
    if (value) {
        *profile |= (1 << index);
    } else {
        *profile &= ~(1 << index);
    }
}

int getAnswer(CompWord profile, int index) {
    return (profile >> index) & 1;
}

int countYesAnswers(CompWord profile) {
    int count = 0;
    for (int i = 0; i < 10; i++) {
        if ((profile >> i) & 1) {
            count++;
        }
    }
```

```
    return count;
}

void printAnswers(CompWord profile) {
    for (int i = 9; i >= 0; i--) {
        printf("%d", (profile >> i) & 1);
    }
    printf("\n");
}

int compareProfiles(CompWord a, CompWord b) {
    int matches = 0;
    for (int i = 0; i < 10; i++) {
        int bitA = (a >> i) & 1;
        int bitB = (b >> i) & 1;
        if (bitA == bitB) {
            matches++;
        }
    }
    return matches;
}

CompWord unionProfiles(CompWord a, CompWord b) {
    return a | b;
}

CompWord intersectProfiles(CompWord a, CompWord b) {
    return a & b;
}

CompWord differenceProfiles(CompWord a, CompWord b) {
    return a & (~b);
}
```

Visualization:

image

Feedback/comment:

Discussion/Explanation:

Item3 - Questionnaire - B.V. Array

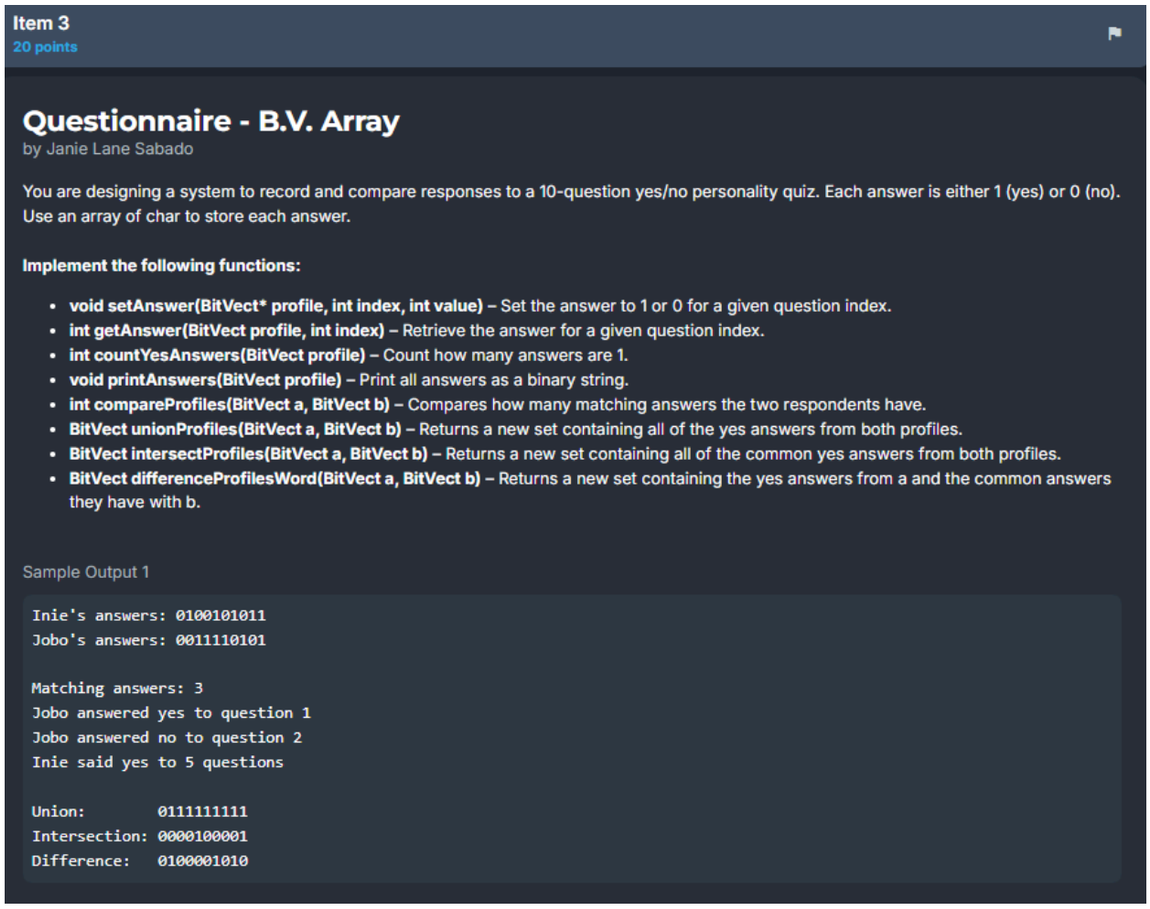
by Janie Lane Sabado

Filename:

[windows] → *filename*

[macOS] → *filename*

Tasks/Instructions:



Item 3
20 points

Questionnaire - B.V. Array

by Janie Lane Sabado

You are designing a system to record and compare responses to a 10-question yes/no personality quiz. Each answer is either 1 (yes) or 0 (no). Use an array of char to store each answer.

Implement the following functions:

- **void setAnswer(BitVect* profile, int index, int value)** – Set the answer to 1 or 0 for a given question index.
- **int getAnswer(BitVect profile, int index)** – Retrieve the answer for a given question index.
- **int countYesAnswers(BitVect profile)** – Count how many answers are 1.
- **void printAnswers(BitVect profile)** – Print all answers as a binary string.
- **int compareProfiles(BitVect a, BitVect b)** – Compares how many matching answers the two respondents have.
- **BitVect unionProfiles(BitVect a, BitVect b)** – Returns a new set containing all of the yes answers from both profiles.
- **BitVect intersectProfiles(BitVect a, BitVect b)** – Returns a new set containing all of the common yes answers from both profiles.
- **BitVect differenceProfilesWord(BitVect a, BitVect b)** – Returns a new set containing the yes answers from a and the common answers they have with b.

Sample Output 1

```
Inie's answers: 0100101011
Jobo's answers: 0011110101

Matching answers: 3
Jobo answered yes to question 1
Jobo answered no to question 2
Inie said yes to 5 questions

Union:        0111111111
Intersection: 0000100001
Difference:   0100001010
```

Test Cases:

item3

Code:

main.c - locked

```c
#include <stdio.h>
#include "bitvector.h"

int main() {
    BitVect inie = {0};
    BitVect jobo = {0};
```

```
    setAnswer(&inie, 0, 1); // Q1: Yes
    setAnswer(&inie, 1, 1); // Q2: Yes
    setAnswer(&inie, 3, 1); // Q4: Yes
    setAnswer(&inie, 5, 1); // Q6: Yes
    setAnswer(&inie, 8, 1); // Q9: Yes
    setAnswer(&jobo, 0, 1); // Q1: Yes
    setAnswer(&jobo, 2, 1); // Q3: Yes
    setAnswer(&jobo, 4, 1); // Q5: Yes
    setAnswer(&jobo, 5, 1); // Q6: Yes
    setAnswer(&jobo, 6, 1); // Q7: Yes
    setAnswer(&jobo, 7, 1); // Q8: Yes

    printf("Inie's answers: ");
    printAnswers(inie);

    printf("Jobo's answers: ");
    printAnswers(jobo);

    printf("\nMatching answers: %d\n", compareProfiles(inie,
jobo));

    printf("Jobo answered %s to question %d\n",(getAnswer(job
o,0))?"yes":"no",1);
    printf("Jobo answered %s to question %d\n",(getAnswer(job
o,1))?"yes":"no",2);
    printf("Inie said yes to %d questions\n\n", countYesAnswe
rs(inie));

    BitVect unionSet = unionProfiles(inie, jobo);
    BitVect intersectSet = intersectProfiles(inie, jobo);
    BitVect diffSet = differenceProfiles(inie, jobo);

    printf("Union:        "); printAnswers(unionSet);
    printf("Intersection: "); printAnswers(intersectSet);
    printf("Difference:   "); printAnswers(diffSet);

    return 0;
}
```

bitvector.h - locked

```
#ifndef BITVECTOR_H
#define BITVECTOR_H

#include <stdio.h>
```

```c
#include <stdbool.h>

typedef struct {
    char answers[10];
} BitVect;

void setAnswer(BitVect* profile, int index, int value);
int getAnswer(BitVect profile, int index);
int countYesAnswers(BitVect profile);
void printAnswers(BitVect profile);
int compareProfiles(BitVect a, BitVect b);

BitVect unionProfiles(BitVect a, BitVect b);
BitVect intersectProfiles(BitVect a, BitVect b);
BitVect differenceProfiles(BitVect a, BitVect b);

#endif // BITVECTOR_H
```

bitvector.c - answer here (raw)

```c
#include <stdio.h>
#include <stdbool.h>
#include "bitvector.h"

// write your funcitons here...
```

bitvector.c - answered code

```c
#include <stdio.h>
#include <stdbool.h>
#include "bitvector.h"

// write your funcitons here...

#define NUM_QUESTIONS 10

void setAnswer(BitVect* profile, int index, int value) {
    if (index < 0 || index >= NUM_QUESTIONS) return;
    profile->answers[index] = value ? 1 : 0;
}

int getAnswer(BitVect profile, int index) {
    if (index < 0 || index >= NUM_QUESTIONS) return 0;
    return profile.answers[index];
}
```

```c
int countYesAnswers(BitVect profile) {
    int count = 0;
    for (int i = 0; i < NUM_QUESTIONS; i++) {
        if (profile.answers[i]) count++;
    }
    return count;
}

void printAnswers(BitVect profile) {
    for (int i = NUM_QUESTIONS - 1; i >= 0; i--) {
        printf("%d", profile.answers[i]);
    }
    printf("\n");
}

int compareProfiles(BitVect a, BitVect b) {
    int matches = 0;
    for (int i = 0; i < NUM_QUESTIONS; i++) {
        if (a.answers[i] == b.answers[i]) matches++;
    }
    return matches;
}

BitVect unionProfiles(BitVect a, BitVect b) {
    BitVect result = {0};
    for (int i = 0; i < NUM_QUESTIONS; i++) {
        result.answers[i] = (a.answers[i] || b.answers[i]) ?
1 : 0;
    }
    return result;
}

BitVect intersectProfiles(BitVect a, BitVect b) {
    BitVect result = {0};
    for (int i = 0; i < NUM_QUESTIONS; i++) {
        result.answers[i] = (a.answers[i] && b.answers[i]) ?
1 : 0;
    }
    return result;
}

BitVect differenceProfiles(BitVect a, BitVect b) {
    BitVect result = {0};
    for (int i = 0; i < NUM_QUESTIONS; i++) {
        result.answers[i] = (a.answers[i] && !b.answers[i]) ?
1 : 0;
    }
```

```
    return result;
}
```

Visualization:

image

Feedback/comment:

Discussion/Explanation:

Item 4 - Bit Vector V3

by Edrian Guanzon

Filename:

[windows] → *filename*

[macOS] → *filename*

Tasks/Instructions:



Test Cases:

Item 4

Code:

main.c - answer here (raw)

```c
#include <stdio.h>
#include <stdbool.h>

#define ARRAY_SIZE 8
typedef bool Set[ARRAY_SIZE];

// Function prototypes
void initSet(Set s);
void addElement(Set s, int elem);
void displaySet(Set s);
void unionSet(Set a, Set b, Set result);
void intersectionSet(Set a, Set b, Set result);
void differenceSet(Set a, Set b, Set result);

// Function definitions
```

```
void initSet(Set s) {

}

void addElement(Set s, int elem) {

}

void displaySet(Set s) {

}

void unionSet(Set a, Set b, Set result) {

}

void intersectionSet(Set a, Set b, Set result) {

}

void differenceSet(Set a, Set b, Set result) {

}

// MAIN PROGRAM
int main() {
    Set A, B, result;

    // Initialize sets
    initSet(A);
    initSet(B);
    initSet(result);

    // A = {1, 3, 5}
    addElement(A, 1);
    addElement(A, 3);
    addElement(A, 5);

    // B = {3, 4, 5}
    addElement(B, 3);
    addElement(B, 4);
    addElement(B, 5);

    // Display sets
    printf("Set A: ");
    displaySet(A);

    printf("Set B: ");
```

```
    displaySet(B);

    // Union
    unionSet(A, B, result);
    printf("Union (A ∪ B): ");
    displaySet(result);

    // Intersection
    intersectionSet(A, B, result);
    printf("Intersection (A ∩ B): ");
    displaySet(result);

    // Difference A - B
    differenceSet(A, B, result);
    printf("Difference (A - B): ");
    displaySet(result);

    return 0;
}
```

main.c - answered code (somewhat wrong)

```
#include <stdio.h>
#include <stdbool.h>

#define ARRAY_SIZE 8
typedef bool Set[ARRAY_SIZE];

// Function prototypes
void initSet(Set s);
void addElement(Set s, int elem);
void displaySet(Set s);
void unionSet(Set a, Set b, Set result);
void intersectionSet(Set a, Set b, Set result);
void differenceSet(Set a, Set b, Set result);

// Function definitions
void initSet(Set s) {
    for(int i=0; i<ARRAY_SIZE; i++){
        s[i] = false;
    }
}

void addElement(Set s, int elem) {
    if(elem>=0 && elem<ARRAY_SIZE){
        s[elem] = true;
```

```
    }
}

void displaySet(Set s) {
    printf("{ ");
    for(int i=0; i<ARRAY_SIZE; i++){
        if(s[i]){
            printf("%d ", i);
        }
    }

}

void unionSet(Set a, Set b, Set result) {
    for(int i=0; i<ARRAY_SIZE; i++){
        result[i] = a[i] || b[i];
    }
}

void intersectionSet(Set a, Set b, Set result) {
    for(int i=0; i<ARRAY_SIZE; i++){
        result[i] = a[i] && b[i];
    }
}

void differenceSet(Set a, Set b, Set result) {
    for(int i=0; i<ARRAY_SIZE; i++){
        result[i] = a[i] && !b[i];
    }
}

// MAIN PROGRAM
int main() {
    Set A, B, result;

    // Initialize sets
    initSet(A);
    initSet(B);
    initSet(result);

    // A = {1, 3, 5}
    addElement(A, 1);
    addElement(A, 3);
    addElement(A, 5);

    // B = {3, 4, 5}
    addElement(B, 3);
    addElement(B, 4);
```

```
    addElement(B, 5);

    // Display sets
    printf("Set A: ");
    displaySet(A);

    printf("Set B: ");
    displaySet(B);

    // Union
    unionSet(A, B, result);
    printf("Union (A ∪ B): ");
    displaySet(result);

    // Intersection
    intersectionSet(A, B, result);
    printf("Intersection (A ∩ B): ");
    displaySet(result);

    // Difference A - B
    differenceSet(A, B, result);
    printf("Difference (A - B): ");
    displaySet(result);

    return 0;
}
```

Visualization:

image

Feedback/comment:

Discussion/Explanation:

Item5 - [AA] - Sabado Cipher - Encoding

by Janie Lane Sabado

Filename:

[windows] → *filename*

[macOS] → *filename*

Tasks/Instructions:

Item 5
30 points

[//] - Sabado Cipher - Encoding

by Janie Lane Sabado

Background:

Janie is going back to Davao for All Soul's day and her boyfriend is there to pick her up from the airport. Her boyfriend asked if there is anything she would want as a gift when she arrives and she decides that she wants to encode the message for her boyfriend. She wants to be able to convert an 8 letter word into a series of 4 integer values. Now you must help Janie in encoding the message.

Objective:

Create the three functions that is necessary in encoding the message in the file `encode.c`

1. `int* encodeString(char string[])` - this function takes the string and returns a bit vector array
2. `short int* arrayToBitmask(int arr[])` - this function takes the bit vector array and returns a set of integers
3. `void displayBitPattern(int arr[])` - this function displays the bit pattern of the bit vector array

Notes:

1. Bit vector array is where each index of the array represents a boolean value: `0` or `1`
2. Remember a short integer is 16 bits, given the characters 'Y' and 'O' is represented as 01011001 and 01001111 respectively, together they represent the short integer 22863

Encoding Guide:

Given the two characters 'Y' and 'O', it is represented in bits as 01011001 and 01001111 respectively. Those bits are then stored in a bit vector array and then converted to represent a short integer value.

```
char letters[2] = {'Y','O'}
```

Encoding Guide:

Given the two characters 'Y' and 'O', it is represented in bits as 01011001 and 01001111 respectively. Those bits are then stored in a bit vector array and then converted to represent a short integer value.

```
char letters[2] = {'Y','O'}

bit representation: 01011001 01001111
     short integer: 22863
```

Sample Output 1

```
=== Sabado Cipher ===

Given the 8 letter word, encode the string into an array of short int
EARRINGS

Enter test case choice:
1. Convert the characters into a bit vector Array
2. Encode the String using the converted bit vector Array
3. Convert and Encode while displaying both

Choice: 1

Bit Pattern:
01000101 01000001 01010010 01010010 01001001 01001110 01000111 01010011
```

Sample Output 2

Sample Output 2

```
=== Sabado Cipher ===

Given the 8 letter word, encode the string into an array of short int
EARRINGS

Enter test case choice:
1. Convert the characters into a bit vector Array
2. Encode the String using the converted bit vector Array
3. Convert and Encode while displaying both

Choice: 3

Bit Pattern:
01000101 01000001 01010010 01010010 01001001 01001110 01000111 01010011

Encoded Message:
17729 21074 18766 18259
```

Test Cases:

item5

Code:

main.c - locked

```c
#include<stdio.h>
#include<stdlib.h>
#include"encode.h"

int main(){
    char string[9]={'E','A','R','R','I','N','G','S','\0'};
    //short int word[4]={22863,21792,16961,19524};
    //short int word[4]={17729,21074,18766,18259};
    int *arr;
    short int *word;
    int x;
    int choice;


    printf("=== Sabado Cipher ===\n\n");
    printf("Given the 8 letter word, encode the string into a
n array of short int\n");
    printf("%s\n\n",string);

    printf("Enter test case choice: \n");
    printf("1. Convert the characters into a bit vector Array
\n");
    printf("2. Encode the String using the converted bit vect
or Array\n");
    printf("3. Convert and Encode while displaying both\n\nCh
oice: ");
```

```
    scanf("%d", &choice);

    printf("\n");
    switch(choice){
        case 1:
            arr=encodeString(string);
            printf("Bit Pattern:\n");
            displayBitPattern(arr);
            free(arr);

            break;

        case 2:
            arr=encodeString(string);
            word=arrayToBitmask(arr);
            printf("\nEncoded Message:\n");

            for(x=0;x<4;x++){
                printf("%d ",word[x]);
            }
            free(word);
            break;

        case 3:
            arr=encodeString(string);
            printf("Bit Pattern:\n");
            displayBitPattern(arr);
            word=arrayToBitmask(arr);
            printf("\n\nEncoded Message:\n");

            for(x=0;x<4;x++){
                printf("%d ",word[x]);
            }

            free(word);
            free(arr);

            break;

        default:
            printf("Invalid Choice");
            break;
    }



    return 0;
```

```
}
```

encode.h - locked

```
#ifndef ENCODE_H
#define ENCODE_H

#include<stdio.h>
#include<stdlib.h>

int* encodeString(char string[]);
short int* arrayToBitmask(int arr[]);
void displayBitPattern(int arr[]);

#endif
```

encode.c - answer here (raw)

```
#include<stdio.h>
#include<stdlib.h>
#include"encode.h"

// Write your functions here...
```

encode.c - answered code

```

```

Visualization:

image

Feedback/comment:

Discussion/Explanation:

Item 6 - [AA] - Sabado Cipher - Decoding

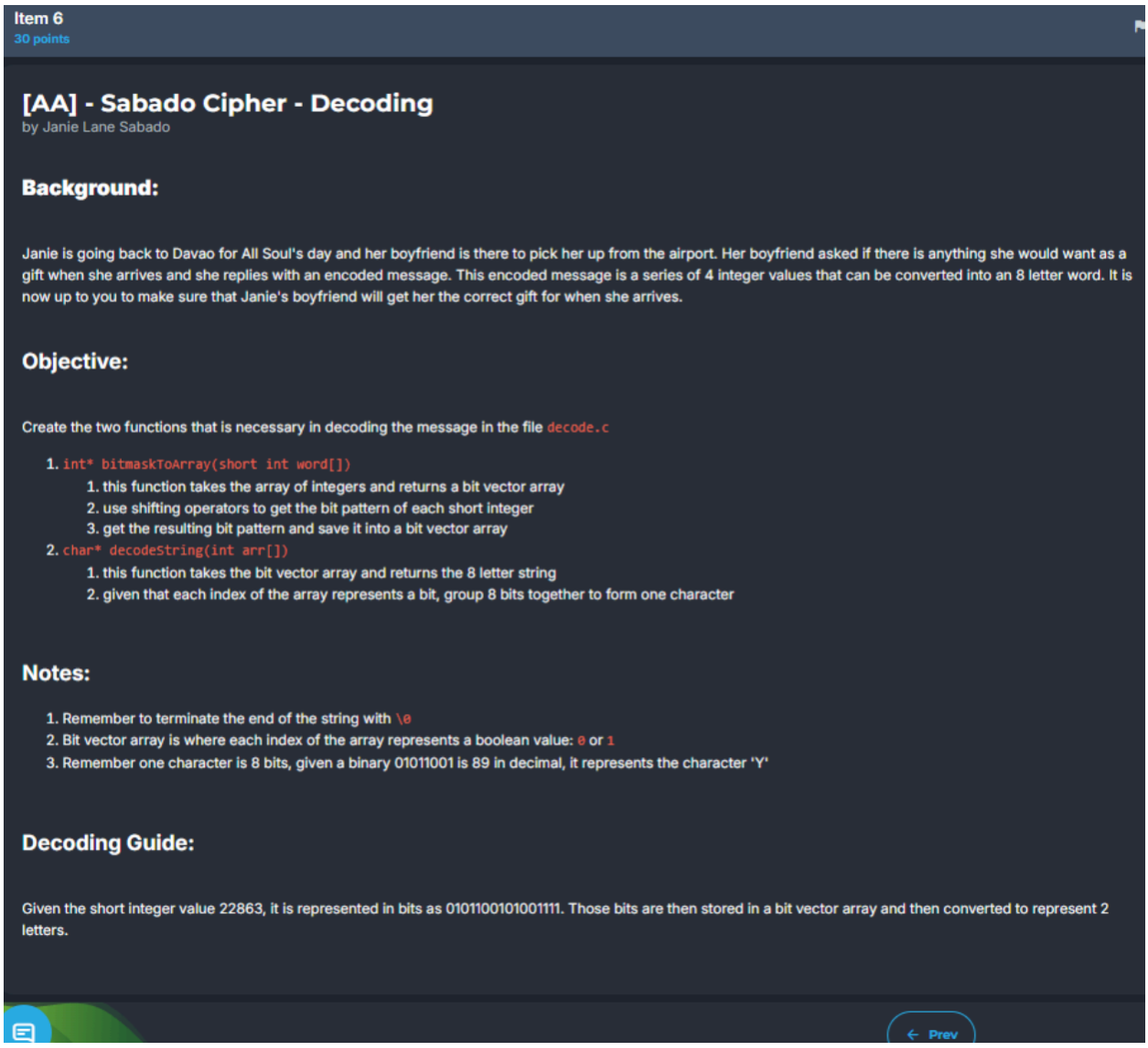
by Janie Lane Sabado

Filename:

[windows] → *filename*

[macOS] → *filename*

Tasks/Instructions:



Item 6
30 points

[AA] - Sabado Cipher - Decoding

by Janie Lane Sabado

Background:

Janie is going back to Davao for All Soul's day and her boyfriend is there to pick her up from the airport. Her boyfriend asked if there is anything she would want as a gift when she arrives and she replies with an encoded message. This encoded message is a series of 4 integer values that can be converted into an 8 letter word. It is now up to you to make sure that Janie's boyfriend will get her the correct gift for when she arrives.

Objective:

Create the two functions that is necessary in decoding the message in the file `decode.c`

1. `int* bitmaskToArray(short int word[])`
    1. this function takes the array of integers and returns a bit vector array
    2. use shifting operators to get the bit pattern of each short integer
    3. get the resulting bit pattern and save it into a bit vector array
2. `char* decodeString(int arr[])`
    1. this function takes the bit vector array and returns the 8 letter string
    2. given that each index of the array represents a bit, group 8 bits together to form one character

Notes:

1. Remember to terminate the end of the string with `\0`
2. Bit vector array is where each index of the array represents a boolean value: `0` or `1`
3. Remember one character is 8 bits, given a binary 01011001 is 89 in decimal, it represents the character 'Y'

Decoding Guide:

Given the short integer value 22863, it is represented in bits as 0101100101001111. Those bits are then stored in a bit vector array and then converted to represent 2 letters.

Prev

Decoding Guide:

Given the short integer value 22863, it is represented in bits as 0101100101001111. Those bits are then stored in a bit vector array and then converted to represent 2 letters.

```
short int val = 22863

bit representation: 0101100101001111
          letters: YO
```

Sample Output 1

```
=== Sabado Cipher ===

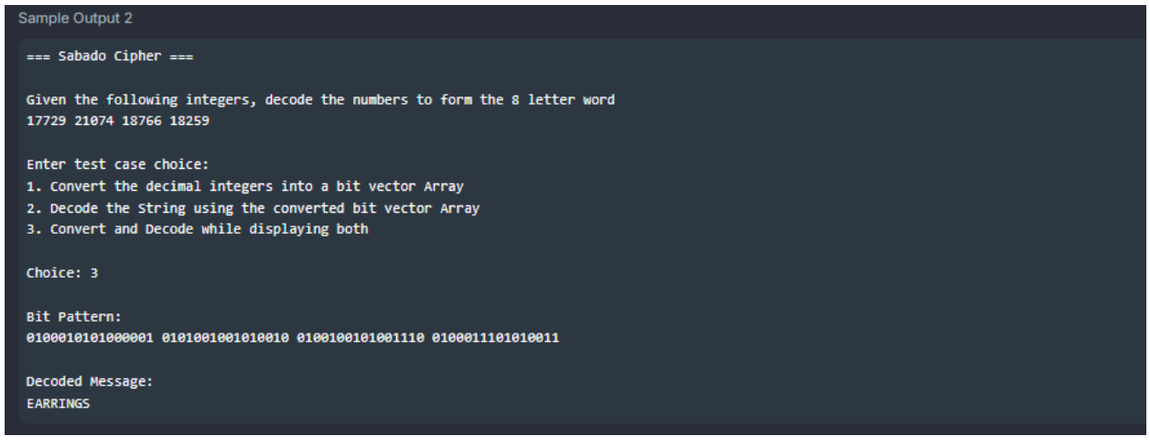
Given the following integers, decode the numbers to form the 8 letter word
17729 21074 18766 18259

Enter test case choice:
1. Convert the decimal integers into a bit vector Array
2. Decode the String using the converted bit vector Array
3. Convert and Decode while displaying both

Choice: 1

Bit Pattern:
0100010101000001 0101001001010010 0100100101001110 0100011101010011
```

Sample Output 2



Sample Output 2

```
=== Sabado Cipher ===

Given the following integers, decode the numbers to form the 8 letter word
17729 21074 18766 18259

Enter test case choice:
1. Convert the decimal integers into a bit vector Array
2. Decode the String using the converted bit vector Array
3. Convert and Decode while displaying both

Choice: 3

Bit Pattern:
0100010101000001 0101001001010010 0100100101001110 0100011101010011

Decoded Message:
EARRINGS
```

Test Cases:

Item 6

Code:

main.c - locked

```c
#include<stdio.h>
#include<stdlib.h>
#include"decode.h"

int main(){
    //short int word[4]={22863,21792,16961,19524};
    short int word[4]={17729,21074,18766,18259};
    int *arr;
    char *string;
    int choice;
```

```
    printf("=== Sabado Cipher ===\n\n");
    printf("Given the following integers, decode the numbers
to form the 8 letter word\n");
    printf("%d %d %d %d\n\n",word[0],word[1],word[2],word
[3]);

    printf("Enter test case choice: \n");
    printf("1. Convert the decimal integers into a bit vector
Array\n");
    printf("2. Decode the String using the converted bit vect
or Array\n");
    printf("3. Convert and Decode while displaying both\n\nCh
oice: ");
    scanf("%d", &choice);

    printf("\n");
    switch(choice){
        case 1:
            arr=bitmaskToArray(word);
            printf("Bit Pattern:\n");
            displayBitPattern(arr);
            free(arr);

            break;

        case 2:
            arr=bitmaskToArray(word);
            string=decodeString(arr);
            printf("\nDecoded Message:");
            printf("\n%s",string);
            free(string);
            break;

        case 3:
            arr=bitmaskToArray(word);
            printf("Bit Pattern:\n");
            displayBitPattern(arr);
            string=decodeString(arr);
            printf("\n\nDecoded Message:");
            printf("\n%s",string);

            free(arr);
            free(string);

            break;
```

```
        default:
            printf("Invalid Choice");
            break;
    }



    return 0;

}


void displayBitPattern(int arr[]){
    int x;

    for(x=0;x<64;x++){
        printf("%d",arr[x]);
        if((x + 1) % 16 == 0 && x != 63){
            printf(" ");
        }
    }

}
```

decode.h - locked

```
#ifndef DECODE_H
#define DECODE_H

#include<stdio.h>
#include<stdlib.h>

int* bitmaskToArray(short int word[]);
char* decodeString(int arr[]);
void displayBitPattern(int arr[]);

#endif
```

decode.c - answer here (raw)

```
#include<stdio.h>
#include<stdlib.h>
#include"decode.h"
```

```c
int* bitmaskToArray(short int word[])
{

}

char* decodeString(int arr[])
{

}
```

decode.c - answered code

```c
#include<stdio.h>
#include<stdlib.h>
#include"decode.h"


int* bitmaskToArray(short int word[])
{
     int* arr = (int*)malloc(64 * sizeof(int));
    if (arr == NULL) return NULL;

    int index = 0;
    for (int i = 0; i < 4; i++) {
        for (int j = 15; j >= 0; j--) {
            arr[index++] = (word[i] >> j) & 1;
        }
    }
    return arr;

}

char* decodeString(int arr[])
{
        char* string = (char*)malloc(9 * sizeof(char));
    if (string == NULL) return NULL;

    for (int i = 0; i < 8; i++) {
        char ch = 0;
        for (int j = 0; j < 8; j++) {
            ch = (ch << 1) | arr[i * 8 + j];
        }
        string[i] = ch;
    }
    string[8] = '\0';
```

```
    return string;

}
```

Visualization:

image

Feedback/comment:

Discussion/Explanation:

DONE